

FTP

The **File Transfer Protocol (FTP)** is one of the oldest protocols on the Internet. The FTP runs within the application layer of the TCP/IP protocol stack. Thus, it is on the same layer as **HTTP** or **POP**. These protocols also work with the support of browsers or email clients to perform their services. There are also special FTP programs for the File Transfer Protocol.

Let us imagine that we want to upload local files to a server and download other files using the **FTP** protocol. In an FTP connection, two channels are opened. First, the client and server establish a control channel through **TCP port 21**. The client sends commands to the server, and the server returns status codes. Then both communication participants can establish the data channel via **TCP port 20**. This channel is used exclusively for data transmission, and the protocol watches for errors during this process. If a connection is broken off during transmission, the transport can be resumed after re-established contact.

A distinction is made between **active** and **passive** FTP. In the active variant, the client establishes the connection as described via TCP port 21 and thus informs the server via which client-side port the server can transmit its responses. However, if a firewall protects the client, the server cannot reply because all external connections are blocked. For this purpose, the **passive mode** has been developed. Here, the server announces a port through which the client can establish the data channel. Since the client initiates the connection in this method, the firewall does not block the transfer.

The FTP knows different **commands** and status codes. Not all of these commands are consistently implemented on the server. For example, the client-side instructs the server-side to upload or download files, organize directories or delete files. The server responds in each case with a status code that indicates whether the command was successfully implemented. A list of possible status codes can be found [here](#).

Usually, we need credentials to use FTP on a server. We also need to know that FTP is a **clear-text** protocol that can sometimes be sniffed if conditions on the network are right. However, there is also the possibility that a server offers **anonymous FTP**. The server operator then allows any user to upload or download files via FTP without using a password. Since there are security risks associated with such a public FTP server, the options for users are usually limited.

TFTP

Trivial File Transfer Protocol (TFTP) is simpler than FTP and performs file transfers between client and server processes. However, it **does not** provide user authentication and other valuable features supported by FTP. In addition, while FTP uses TCP, TFTP uses **UDP**, making it an unreliable protocol and causing it to use UDP-assisted application layer recovery.

This is reflected, for example, in the fact that TFTP, unlike FTP, does not require the user's authentication. It does not support protected login via passwords and sets limits on access based solely on the read and write permissions of a file in the operating system. Practically, this leads to TFTP operating exclusively in directories and with files that have been shared with all users and can be read and written globally. Because of the lack of security, TFTP, unlike FTP, may only be used in local and protected networks.

Let us take a look at a few commands of **TFTP**:

Commands	Description
connect	Sets the remote host, and optionally the port, for file transfers.
get	Transfers a file or set of files from the remote host to the local host.
put	Transfers a file or set of files from the local host onto the remote host.
quit	Exits tftp.
status	Shows the current status of tftp, including the current transfer mode (ascii or binary), connection status, time-out value, and so on.

Commands	Description
<code>verbose</code>	Turns verbose mode, which displays additional information during file transfer, on or off.

Unlike the FTP client, `TFTP` does not have directory listing functionality.

Default Configuration

One of the most used FTP servers on Linux-based distributions is `vsFTPD`. The default configuration of `vsFTPD` can be found in `/etc/vsftpd.conf`, and some settings are already predefined by default. It is highly recommended to install the `vsFTPD` server on a VM and have a closer look at this configuration.

Install vsFTPD

Install vsFTPD
<code>dbcypth0n@htb[/htb]\$ sudo apt install vsftpd</code>

The `vsFTPD` server is only one of a few FTP servers available to us. There are many different alternatives to it, which also bring, among other things, many more functions and configuration options with them. We will use the `vsFTPD` server because it is an excellent way to show the configuration possibilities of an FTP server in a simple and easy-to-understand way without going into the details of the man pages. If we look at the configuration file of `vsFTPD`, we will see many options and settings that are either commented or commented out. However, the configuration file does not contain all possible settings that can be made. The existing and missing ones can be found on the [man page](#).

vsFTPD Config File

vsFTPD Config File
<code>dbcypth0n@htb[/htb]\$ cat /etc/vsftpd.conf grep -v "#"</code>

Setting	Description
<code>listen=NO</code>	Run from inetd or as a standalone daemon?
<code>listen_ipv6=YES</code>	Listen on IPv6 ?
<code>anonymous_enable=NO</code>	Enable Anonymous access?
<code>local_enable=YES</code>	Allow local users to login?
<code>dirmessage_enable=YES</code>	Display active directory messages when users go into certain directories?
<code>use_localtime=YES</code>	Use local time?
<code>xferlog_enable=YES</code>	Activate logging of uploads/downloads?
<code>connect_from_port_20=YES</code>	Connect from port 20?
<code>secure_chroot_dir=/var/run/vsftpd/empty</code>	Name of an empty directory
<code>pam_service_name=vsftpd</code>	This string is the name of the PAM service vsftpd will use.
<code>rsa_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem</code>	The last three options specify the location of the RSA certificate to use for SSL encrypted connections.

Setting	Description
<code>rsa_private_key_file=/etc/ssl/private/ssl-cert-snakeoil.key</code>	
<code>ssl_enable=NO</code>	

In addition, there is a file called `/etc/ftputers` that we also need to pay attention to, as this file is used to deny certain users access to the FTP service. In the following example, the users `guest`, `john`, and `kevin` are not permitted to log in to the FTP service, even if they exist on the Linux system.

FTPUSERS

FTPUSERS
<pre>dbcyp0n@htb[/htb]\$ cat /etc/ftputers guest john kevin</pre>

Dangerous Settings

There are many different security-related settings we can make on each FTP server. These can have various purposes, such as testing connections through the firewalls, testing routes, and authentication mechanisms. One of these authentication mechanisms is the `anonymous` user. This is often used to allow everyone on the internal network to share files and data without accessing each other's computers. With vsFTPD, the `optional settings` that can be added to the configuration file for the anonymous login look like this:

Setting	Description
<code>anonymous_enable=YES</code>	Allowing anonymous login?
<code>anon_upload_enable=YES</code>	Allowing anonymous to upload files?
<code>anon_mkdir_write_enable=YES</code>	Allowing anonymous to create new directories?
<code>no_anon_password=YES</code>	Do not ask anonymous for password?
<code>anon_root=/home/username/ftp</code>	Directory for anonymous.
<code>write_enable=YES</code>	Allow the usage of FTP commands: STOR, DELE, RNFR, RNTD, MKD, RMD, APPE, and SITE?

With the standard FTP client (`ftp`), we can access the FTP server accordingly and log in with the anonymous user if the settings shown above have been used. The use of the anonymous account can occur in internal environments and infrastructures where the participants are all known. Access to this type of service can be set temporarily or with the setting to accelerate the exchange of files.

As soon as we connect to the vsFTPD server, the `response code 220` is displayed with the banner of the FTP server. Often this banner contains the description of the `service` and even the `version` of it. It also tells us what type of system the FTP server is. One of the most common configurations of FTP servers is to allow `anonymous` access, which does not require legitimate credentials but provides access to some files. Even if we cannot download them, sometimes just listing the contents is enough to generate further ideas and note down information that will help us in another approach.

Anonymous Login

Anonymous Login

```
dbcyp0n@htb[/htb]$ ftp 10.129.14.136

Connected to 10.129.14.136.
220 "Welcome to the HTB Academy vsFTP service."
Name (10.129.14.136:cry0l1t3): anonymous

230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.

ftp> ls

200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-rw-r-- 1 1002 1002 8138592 Sep 14 16:54 Calender.pptx
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Clients
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Documents
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Employees
-rw-rw-r-- 1 1002 1002 41 Sep 14 16:45 Important Notes.txt
226 Directory send OK.
```

However, to get the first overview of the server's settings, we can use the following command:

vsFTPD Status

vsFTPD Status

```
ftp> status

Connected to 10.129.14.136.
No proxy connection.
Connecting using address family: any.
Mode: stream; Type: binary; Form: non-print; Structure: file
Verbose: on; Bell: off; Prompting: on; Globbing: on
Store unique: off; Receive unique: off
Case: off; CR stripping: on
Quote control characters: on
Ntrans: off
Nmap: off
Hash mark printing: off; Use of PORT cmds: on
Tick counter printing: off
```

Some commands should be used occasionally, as these will make the server show us more information that we can use for our purposes. These commands include **debug** and **trace**.

vsFTPD Detailed Output

vsFTPD Detailed Output

```
ftp> debug

Debugging on (debug=1).

ftp> trace

Packet tracing on.

ftp> ls

---> PORT 10,10,14,4,188,195
200 PORT command successful. Consider using PASV.
---> LIST
150 Here comes the directory listing.
-rw-rw-r-- 1 1002 1002 8138592 Sep 14 16:54 Calender.pptx
drwxrwxr-x 2 1002 1002 4096 Sep 14 17:03 Clients
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Documents
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Employees
-rw-rw-r-- 1 1002 1002 41 Sep 14 16:45 Important Notes.txt
226 Directory send OK.
```

Setting	Description
<code>dirmessage_enable=YES</code>	Show a message when they first enter a new directory?
<code>chown_uploads=YES</code>	Change ownership of anonymously uploaded files?
<code>chown_username=username</code>	User who is given ownership of anonymously uploaded files.
<code>local_enable=YES</code>	Enable local users to login?
<code>chroot_local_user=YES</code>	Place local users into their home directory?
<code>chroot_list_enable=YES</code>	Use a list of local users that will be placed in their home directory?

Setting	Description
<code>hide_ids=YES</code>	All user and group information in directory listings will be displayed as "ftp".
<code>ls_recurse_enable=YES</code>	Allows the use of recurse listings.

In the following example, we can see that if the `hide_ids=YES` setting is present, the UID and GUID representation of the service will be overwritten, making it more difficult for us to identify with which rights these files are written and uploaded.

Hiding IDs - YES

Hiding IDs - YES

```

ftp> ls

---> TYPE A
200 Switching to ASCII mode.
ftp: setsockopt (ignored): Permission denied
---> PORT 10,10,14,4,223,101
200 PORT command successful. Consider using PASV.
---> LIST
150 Here comes the directory listing.
-rw-rw-r-- 1 ftp ftp 8138592 Sep 14 16:54 Calender.pptx
drwxrwxr-x 2 ftp ftp 4096 Sep 14 17:03 Clients
drwxrwxr-x 2 ftp ftp 4096 Sep 14 16:50 Documents
drwxrwxr-x 2 ftp ftp 4096 Sep 14 16:50 Employees
-rw-rw-r-- 1 ftp ftp 41 Sep 14 16:45 Important Notes.txt
-rw----- 1 ftp ftp 0 Sep 15 14:57 testupload.txt
226 Directory send OK.

```

This setting is a security feature to prevent local usernames from being revealed. With the usernames, we could attack the services like FTP and SSH and many others with a brute-force attack in theory. However, in reality, [fail2ban](#) solutions are now a standard implementation of any infrastructure that logs the IP address and blocks all access to the infrastructure after a certain number of failed login attempts.

Another helpful setting we can use for our purposes is the `ls_recurse_enable=YES`. This is often set on the vsFTPD server to have a better overview of the FTP directory structure, as it allows us to see all the visible content at once.

Recursive Listing

Recursive Listing

```

ftp> ls -R

---> PORT 10,10,14,4,222,149
200 PORT command successful. Consider using PASV.
---> LIST -R
150 Here comes the directory listing.
.:
-rw-rw-r-- 1 ftp ftp 8138592 Sep 14 16:54 Calender.pptx
drwxrwxr-x 2 ftp ftp 4096 Sep 14 17:03 Clients
drwxrwxr-x 2 ftp ftp 4096 Sep 14 16:50 Documents
drwxrwxr-x 2 ftp ftp 4096 Sep 14 16:50 Employees
-rw-rw-r-- 1 ftp ftp 41 Sep 14 16:45 Important Notes.txt
-rw----- 1 ftp ftp 0 Sep 15 14:57 testupload.txt

./Clients:
drwx----- 2 ftp ftp 4096 Sep 16 18:04 HackTheBox
drwxrwxrwx 2 ftp ftp 4096 Sep 16 18:00 Inlanefreight

./Clients/HackTheBox:
-rw-r--r-- 1 ftp ftp 34872 Sep 16 18:04 appointments.xlsx
-rw-r--r-- 1 ftp ftp 498123 Sep 16 18:04 contract.docx
-rw-r--r-- 1 ftp ftp 478237 Sep 16 18:04 contract.pdf
-rw-r--r-- 1 ftp ftp 348 Sep 16 18:04 meetings.txt

./Clients/Inlanefreight:
-rw-r--r-- 1 ftp ftp 14211 Sep 16 18:00 appointments.xlsx
-rw-r--r-- 1 ftp ftp 37882 Sep 16 17:58 contract.docx
-rw-r--r-- 1 ftp ftp 89 Sep 16 17:58 meetings.txt
-rw-r--r-- 1 ftp ftp 483293 Sep 16 17:59 proposal.pptx

./Documents:
-rw-r--r-- 1 ftp ftp 23211 Sep 16 18:05 appointments-template.xlsx
-rw-r--r-- 1 ftp ftp 32521 Sep 16 18:05 contract-template.docx
-rw-r--r-- 1 ftp ftp 453312 Sep 16 18:05 contract-template.pdf

./Employees:
226 Directory send OK.

```

Downloading files from such an FTP server is one of the main features, as well as uploading files created by us. This allows us, for example, to use LFI vulnerabilities to make the host execute system commands. Apart from the files, we can view, download and inspect. Attacks are also possible with the FTP logs, leading to Remote Command Execution (RCE). This applies to the FTP services and all those we can detect during our enumeration phase.

Download a File

Download a File

```
ftp> ls

200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rwxrwxrwx   1 ftp      ftp           0 Sep 16 17:24 Calendar.pptx
drwxrwxrwx   4 ftp      ftp        4096 Sep 16 17:57 Clients
drwxrwxrwx   2 ftp      ftp        4096 Sep 16 18:05 Documents
drwxrwxrwx   2 ftp      ftp        4096 Sep 16 17:24 Employees
-rwxrwxrwx   1 ftp      ftp         41 Sep 18 15:58 Important Notes.txt
226 Directory send OK.

ftp> get Important\ Notes.txt

local: Important Notes.txt remote: Important Notes.txt
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for Important Notes.txt (41 bytes).
226 Transfer complete.
41 bytes received in 0.00 secs (606.6525 kB/s)

ftp> exit

221 Goodbye.
```

Download a File

```
dbcypH0n@htb[/htb]$ ls | grep Notes.txt

'Important Notes.txt'
```

We also can download all the files and folders we have access to at once. This is especially useful if the FTP server has many different files in a larger folder structure. However, this can cause alarms because no one from the company usually wants to download all files and content all at once.

Download All Available Files

Download All Available Files

```

dbcypth0n@htb[/htb]$ wget -m --no-passive ftp://anonymous:anonymous@10.129.14.136

--2021-09-19 14:45:58-- ftp://anonymous:*password*@10.129.14.136/
=> '10.129.14.136/.listing'
Connecting to 10.129.14.136:21... connected.
Logging in as anonymous ... Logged in!
==> SYST ... done.      ==> PWD ... done.
==> TYPE I ... done.    ==> CWD not needed.
==> PORT ... done.     ==> LIST ... done.
12.12.1.136/.listing          [ <=> ]      466  --.-KB/s    in 0s

2021-09-19 14:45:58 (65,8 MB/s) - '10.129.14.136/.listing' saved [466]
--2021-09-19 14:45:58-- ftp://anonymous:*password*@10.129.14.136/Calendar.pptx
=> '10.129.14.136/Calendar.pptx'
==> CWD not required.
==> SIZE Calendar.pptx ... done.
==> PORT ... done.     ==> RETR Calendar.pptx ... done.

...SNIP...

2021-09-19 14:45:58 (48,3 MB/s) - '10.129.14.136/Employees/.listing' saved [119]

FINISHED --2021-09-19 14:45:58--
Total wall clock time: 0,03s
Downloaded: 15 files, 1,7K in 0,001s (3,02 MB/s)

```

Once we have downloaded all the files, **wget** will create a directory with the name of the IP address of our target. All downloaded files are stored there, which we can then inspect locally.

Download All Available Files

```

dbcypth0n@htb[/htb]$ tree .

.
├── 10.129.14.136
│   ├── Calendar.pptx
│   ├── Clients
│   │   └── Inlanefreight
│   │       ├── appointments.xlsx
│   │       ├── contract.docx
│   │       ├── meetings.txt
│   │       └── proposal.pptx
│   ├── Documents
│   │   ├── appointments-template.xlsx
│   │   ├── contract-template.docx
│   │   └── contract-template.pdf
│   ├── Employees
│   └── Important Notes.txt

```

5 directories, 9 files

Next, we can check if we have the permissions to upload files to the FTP server. Especially with web servers, it is common that files are synchronized, and the developers have quick access to the files. FTP is often used for this purpose, and most of the time, configuration errors are found on servers that the administrators think are not discoverable. The attitude that internal network components cannot be accessed from the outside means that the hardening of internal systems is often neglected and leads to misconfigurations.

The ability to upload files to the FTP server connected to a web server increases the likelihood of gaining direct access to the webserver and even a reverse shell that allows us to execute internal system commands and perhaps even escalate our privileges.

Upload a File

Upload a File

```

dbcypth0n@htb[/htb]$ touch testupload.txt

```


With the **PUT** command, we can upload files in the current folder to the FTP server.

Upload a File

```
ftp> put testupload.txt

local: testupload.txt remote: testupload.txt
---> PORT 10,10,14,4,184,33
200 PORT command successful. Consider using PASV.
---> STOR testupload.txt
150 Ok to send data.
226 Transfer complete.

ftp> ls

---> TYPE A
200 Switching to ASCII mode.
---> PORT 10,10,14,4,223,101
200 PORT command successful. Consider using PASV.
---> LIST
150 Here comes the directory listing.
-rw-rw-r-- 1 1002 1002 8138592 Sep 14 16:54 Calender.pptx
drwxrwxr-x 2 1002 1002 4096 Sep 14 17:03 Clients
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Documents
drwxrwxr-x 2 1002 1002 4096 Sep 14 16:50 Employees
-rw-rw-r-- 1 1002 1002 41 Sep 14 16:45 Important Notes.txt
-rw----- 1 1002 133 0 Sep 15 14:57 testupload.txt
226 Directory send OK.
```

Footprinting the Service

Footprinting using various network scanners is also a handy and widespread approach. These tools make it easier for us to identify different services, even if they are not accessible on standard ports. One of the most widely used tools for this purpose is Nmap. Nmap also brings the **Nmap Scripting Engine (NSE)**, a set of many different scripts written for specific services. More information on the capabilities of Nmap and NSE can be found in the **Network Enumeration with Nmap** module. We can update this database of NSE scripts with the command shown.

Nmap FTP Scripts

Nmap FTP Scripts

```
dbcypb0n@htb[/htb]$ sudo nmap --script-updatedb

Starting Nmap 7.80 ( https://nmap.org ) at 2021-09-19 13:49 CEST
NSE: Updating rule database.
NSE: Script Database updated successfully.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.28 seconds
```

All the NSE scripts are located on the Pwnbox in **/usr/share/nmap/scripts/**, but on our systems, we can find them using a simple command on our system.

Nmap FTP Scripts

```
dbcypth0n@htb[/htb]$ find / -type f -name ftp* 2>/dev/null | grep scripts

/usr/share/nmap/scripts/ftp-syst.nse
/usr/share/nmap/scripts/ftp-vsftpd-backdoor.nse
/usr/share/nmap/scripts/ftp-vuln-cve2010-4221.nse
/usr/share/nmap/scripts/ftp-proftpd-backdoor.nse
/usr/share/nmap/scripts/ftp-bounce.nse
/usr/share/nmap/scripts/ftp-libopie.nse
/usr/share/nmap/scripts/ftp-anon.nse
/usr/share/nmap/scripts/ftp-brute.nse
```

As we already know, the FTP server usually runs on the standard TCP port 21, which we can scan using Nmap. We also use the version scan (**-sV**), aggressive scan (**-A**), and the default script scan (**-sC**) against our target **10.129.14.136**.

Nmap

Nmap

```
dbcypth0n@htb[/htb]$ sudo nmap -sV -p21 -sC -A 10.129.14.136

Starting Nmap 7.80 ( https://nmap.org ) at 2021-09-16 18:12 CEST
Nmap scan report for 10.129.14.136
Host is up (0.00013s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.0.8 or later
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| -rw-rw-rw-rw- 1 ftp      ftp      8138592 Sep 16 17:24 Calendar.pptx [NSE: writeable]
| -rw-rw-rw-rw- 4 ftp      ftp      4096 Sep 16 17:57 Clients [NSE: writeable]
| -rw-rw-rw-rw- 2 ftp      ftp      4096 Sep 16 18:05 Documents [NSE: writeable]
| -rw-rw-rw-rw- 2 ftp      ftp      4096 Sep 16 17:24 Employees [NSE: writeable]
| -rw-rw-rw-rw- 1 ftp      ftp      41 Sep 16 17:24 Important Notes.txt [NSE: writeable]
|_-rw-rw-rw-rw- 1 ftp      ftp      0 Sep 15 14:57 testupload.txt [NSE: writeable]
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to 10.10.14.4
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 2
|   vsFTPd 3.0.3 - secure, fast, stable
|_End of status
```

The default script scan is based on the services' fingerprints, responses, and standard ports. Once Nmap has detected the service, it executes the marked scripts one after the other, providing different information. For example, the **ftp-anon** NSE script checks whether the FTP server allows anonymous access. If so, the contents of the FTP root directory are rendered for the anonymous user.

The **ftp-syst**, for example, executes the **STAT** command, which displays information about the FTP server status. This includes configurations as well as the version of the FTP server. Nmap also provides the ability to trace the progress of NSE scripts at the network level if we use the **--script-trace** option in our scans. This lets us see what commands Nmap sends, what ports are used, and what responses we receive from the scanned server.

Nmap Script Trace

Nmap Script Trace

```
dbcyph0n@htb[/htb]$ sudo nmap -sV -p21 -sC -A 10.129.14.136 --script-trace

Starting Nmap 7.80 ( https://nmap.org ) at 2021-09-19 13:54 CEST
NSOCK INFO [11.4640s] nsock_trace_handler_callback(): Callback: CONNECT SUCCESS for EID 8 [10.129.14.136:21]
NSOCK INFO [11.4640s] nsock_trace_handler_callback(): Callback: CONNECT SUCCESS for EID 16 [10.129.14.136:21]
NSOCK INFO [11.4640s] nsock_trace_handler_callback(): Callback: CONNECT SUCCESS for EID 24 [10.129.14.136:21]
NSOCK INFO [11.4640s] nsock_trace_handler_callback(): Callback: CONNECT SUCCESS for EID 32 [10.129.14.136:21]
NSOCK INFO [11.4640s] nsock_read(): Read request from IOD #1 [10.129.14.136:21] (timeout: 7000ms) EID 42
NSOCK INFO [11.4640s] nsock_read(): Read request from IOD #2 [10.129.14.136:21] (timeout: 9000ms) EID 50
NSOCK INFO [11.4640s] nsock_read(): Read request from IOD #3 [10.129.14.136:21] (timeout: 7000ms) EID 58
NSOCK INFO [11.4640s] nsock_read(): Read request from IOD #4 [10.129.14.136:21] (timeout: 11000ms) EID 66
NSE: TCP 10.10.14.4:54226 > 10.129.14.136:21 | CONNECT
NSE: TCP 10.10.14.4:54228 > 10.129.14.136:21 | CONNECT
NSE: TCP 10.10.14.4:54230 > 10.129.14.136:21 | CONNECT
NSE: TCP 10.10.14.4:54232 > 10.129.14.136:21 | CONNECT
NSOCK INFO [11.4660s] nsock_trace_handler_callback(): Callback: READ SUCCESS for EID 50 [10.129.14.136:21] (41 by
NSOCK INFO [11.4660s] nsock_trace_handler_callback(): Callback: READ SUCCESS for EID 58 [10.129.14.136:21] (41 by
NSE: TCP 10.10.14.4:54228 < 10.129.14.136:21 | 220 Welcome to HTB-Academy FTP service.
```

The scan history shows that four different parallel scans are running against the service, with various timeouts. For the NSE scripts, we see that our local machine uses other output ports (54226, 54228, 54230, 54232) and first initiates the connection with the **CONNECT** command. From the first response from the server, we can see that we are receiving the banner from the server to our second NSE script (54228) from the target FTP server. If necessary, we can, of course, use other applications such as **netcat** or **telnet** to interact with the FTP server.

Service Interaction

Service Interaction

```
dbcyph0n@htb[/htb]$ nc -nv 10.129.14.136 21
```

Service Interaction

```
dbcyph0n@htb[/htb]$ telnet 10.129.14.136 21
```

It looks slightly different if the FTP server runs with TLS/SSL encryption. Because then we need a client that can handle TLS/SSL. For this, we can use the client **openssl** and communicate with the FTP server. The good thing about using **openssl** is that we can see the SSL certificate, which can also be helpful.

Service Interaction

```
dbcyp0n@htb[/htb]$ openssl s_client -connect 10.129.14.136:21 -starttls ftp

CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 C = US, ST = California, L = Sacramento, O = Inlanefreight, OU = Dev, CN = master.inlanefreight.htb, email=master@inlanefreight.htb
verify error:num=18:self signed certificate
verify return:1

depth=0 C = US, ST = California, L = Sacramento, O = Inlanefreight, OU = Dev, CN = master.inlanefreight.htb, email=master@inlanefreight.htb
verify return:1
---
Certificate chain
 0 s:C = US, ST = California, L = Sacramento, O = Inlanefreight, OU = Dev, CN = master.inlanefreight.htb, email=master@inlanefreight.htb
   i:C = US, ST = California, L = Sacramento, O = Inlanefreight, OU = Dev, CN = master.inlanefreight.htb, email=master@inlanefreight.htb
  ---
Server certificate

-----BEGIN CERTIFICATE-----
MIIENTCCAx2gAwIBAgIUD+SLFZAWzX5yLs2q3ZcfdSRQqMYwDQYJKoZIhvcNAQEL
...SNIP...
```

This is because the SSL certificate allows us to recognize the **hostname**, for example, and in most cases also an **email address** for the organization or company. In addition, if the company has several locations worldwide, certificates can also be created for specific locations, which can also be identified using the SSL certificate.

VPN Servers

 Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

UDP 1337 ☒ TCP 443

DOWNLOAD VPN CONNECTION FILE

Start Instance

∞ / 1 spawns left

Waiting to start..

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: 10.129.202.5
Life Left: 38 minutes

Cheat Sheet

Download VPN Connection File

+ 0 Which version of the FTP server is running on the target system? Submit the entire banner as the answer.

InFreight FTP v1.1

Submit

+ 0 Enumerate the FTP server and find the flag.txt file. Submit the contents of it as the answer.

HTB{b7skjr4c76zhds7fzhd4k3ujg7nhdjre}

Submit

PreviousNext

Mark Complete & Next

Cheat Sheet

Resources

Go to Questions

Table of Contents

Introduction

Enumeration Principles	✓
Enumeration Methodology	✓

Infrastructure Based Enumeration

Domain Information	✓
Cloud Resources	✓
Staff	✓

Host Based Enumeration

FTP	✓
SMB	
NFS	

-  DNS
-  SMTP
-  IMAP / POP3
-  SNMP
-  MySQL
-  MSSQL
-  Oracle TNS
-  IPMI

Remote Management Protocols

- Linux Remote Management Protocols
- Windows Remote Management Protocols

Skills Assessment

-  Footprinting Lab - Easy
-  Footprinting Lab - Medium
-  Footprinting Lab - Hard

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left